

Building Reusable Memory Interfaces for Large Language Models



UNIVERSITY
OF TURKU



**Mingyuan Li, Guangsheng Yu, Juyuan Zhang, Xu Wang,
Haonan Zhang, Shaoxiong Ji**



UNIVERSITY
OF TURKU



CONTENTS



PART 1

Motivation



PART 2

XMemory



PART 3

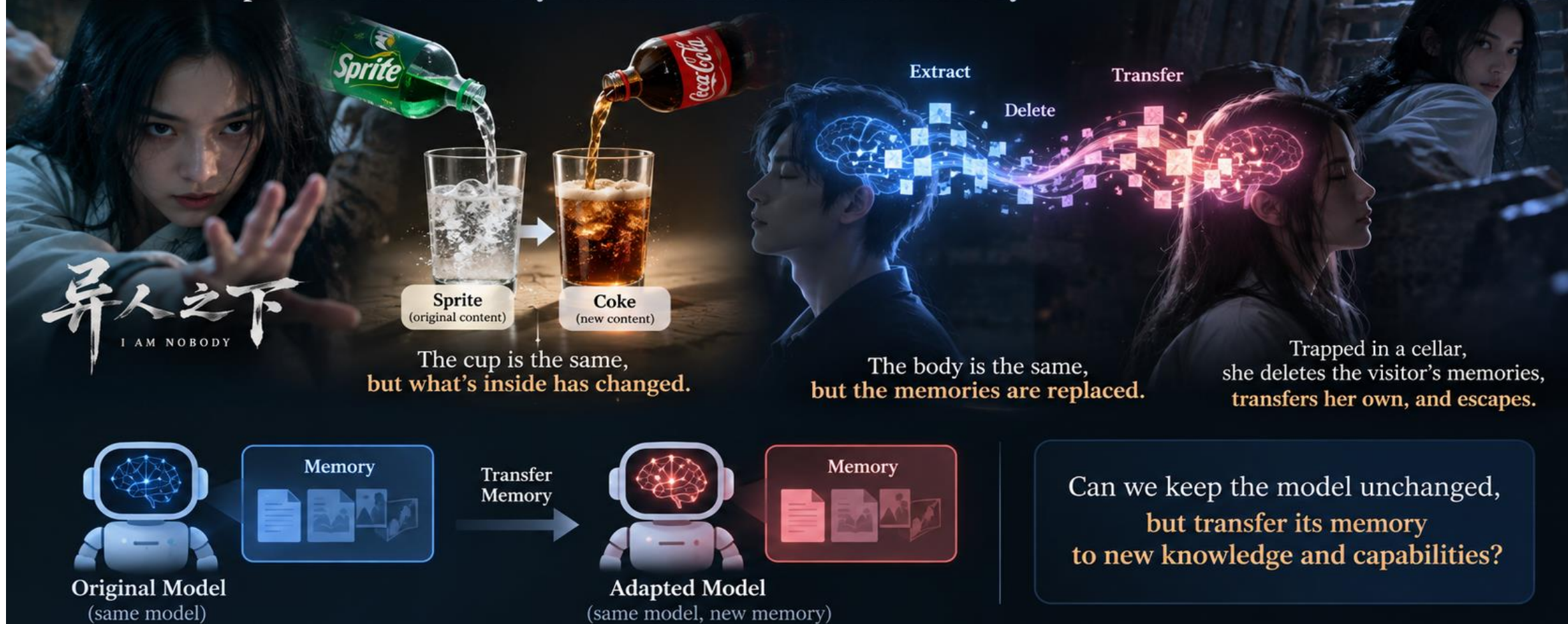
MemoryAthena





What if memories could be poured out and poured in?

From the “Sprite to Coke tce Theory” to Transferable Model Memory



Non-Parametric Memory

Knowledge is stored **outside the model parameters** and accessed at inference time.

- **RAG**: Retrieves relevant documents or passages, then puts them into the prompt.
- **kNN-LM**: Retrieves similar hidden states from a datastore and uses their next-token labels.

Takeaway:

Non-parametric memory is flexible and editable, but the model must retrieve and reuse it every time.

Parametric Memory

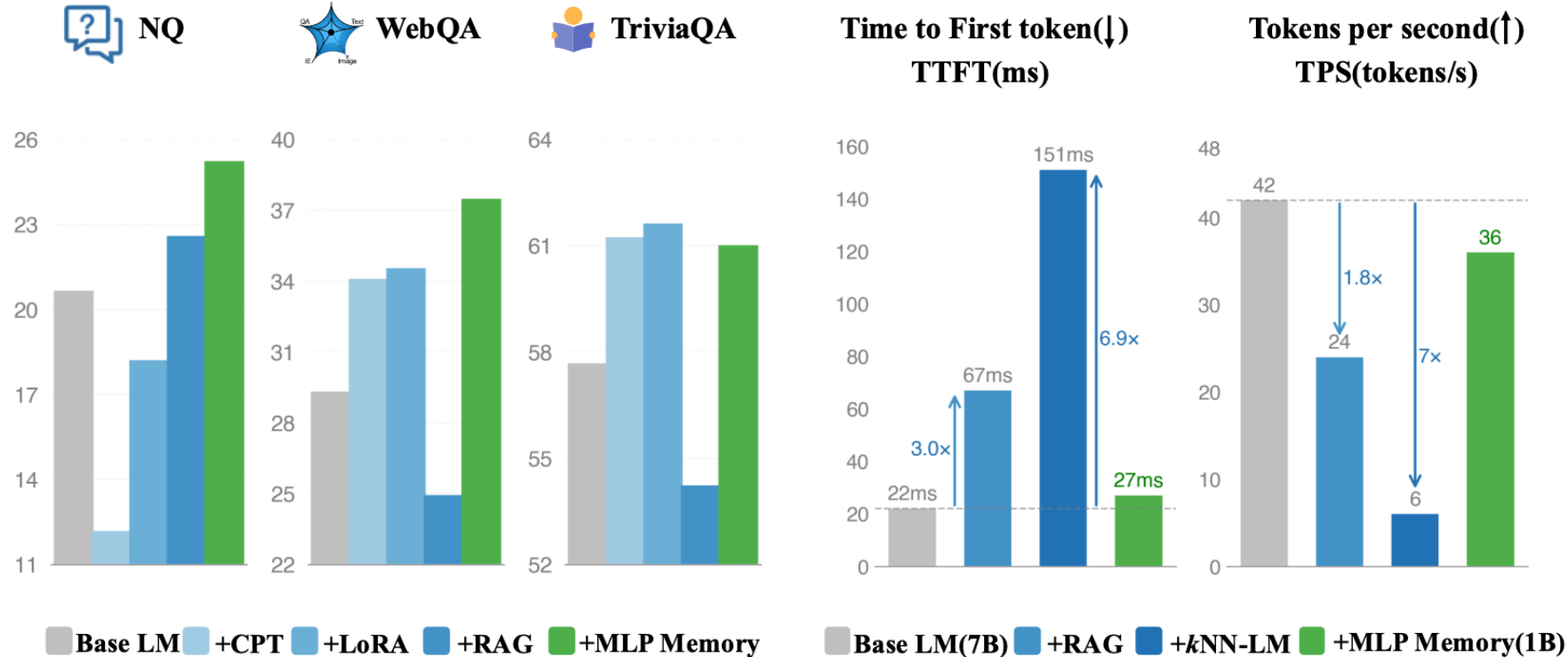
Knowledge is stored inside learned parameters and used directly during inference.

- **CPT:** Continues pre-training the full backbone on new data.
- **LoRA:** Stores new knowledge in lightweight adapter weights.
- **MLP Memory:** Stores retrieval-like knowledge in a learned neural module.

Takeaway:

Parametric memory is efficient to use, but knowledge becomes entangled with learned representations.

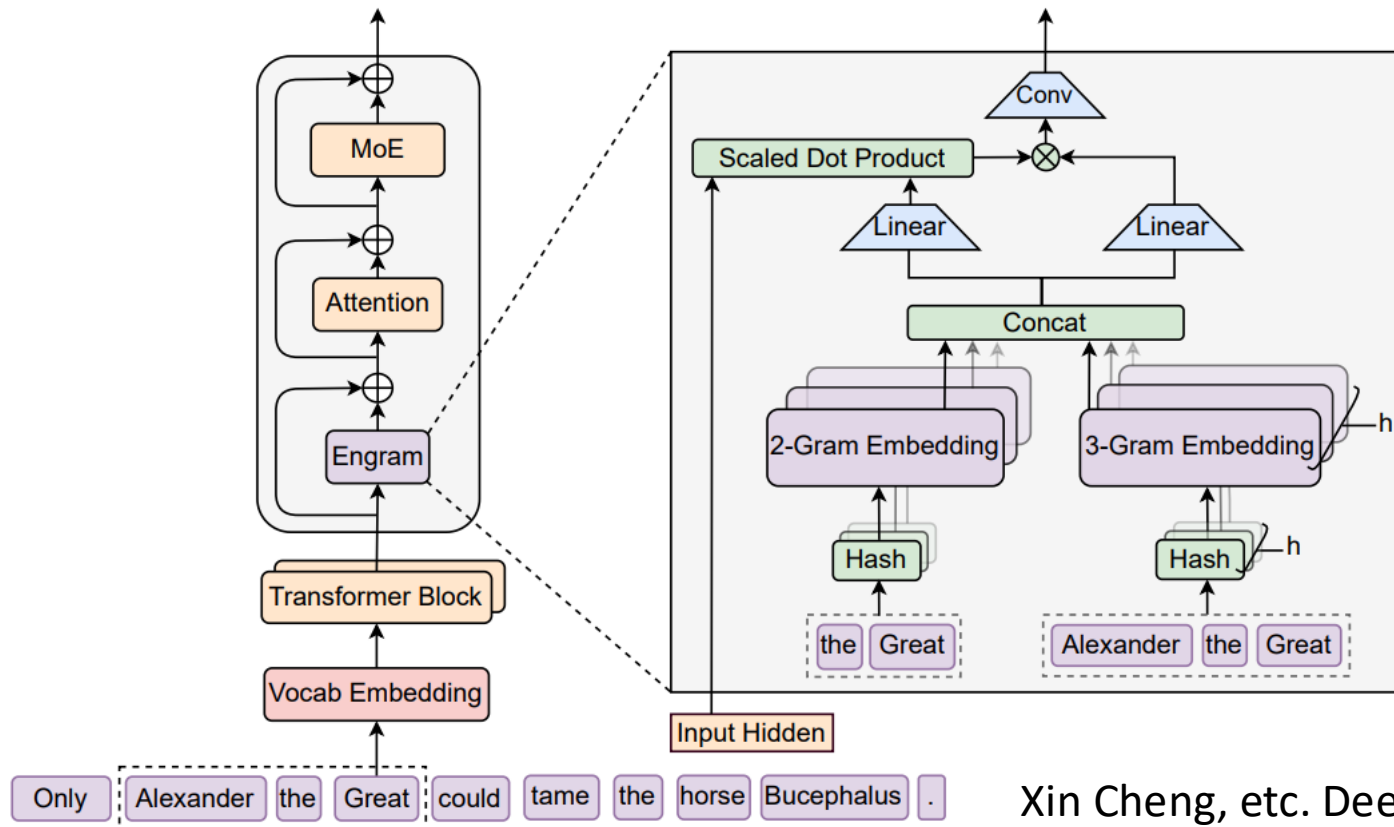
MLP Memory: A Retriever-Pretrained Memory for Large Language Models. Rubin Wei, etc. ICLR, 2026.



This motivates a new direction: instead of storing knowledge entirely in the backbone or repeatedly retrieving raw information, can we build a dedicated, addressable neural memory?

Engram: Conditional Memory via Scalable Lookup

Transformers should not use expensive neural computation to memorize everything
Engram adds a conditional memory module that performs fast **N-gram-based lookup**.



Takeaway:

Engram makes memory an architectural primitive: not raw-text retrieval, not full parameter adaptation, but sparse neural lookup.

Xin Cheng, etc. Deepseek, 2026.



CONTENTS



PART 1

Motivation



PART 2

XMemory



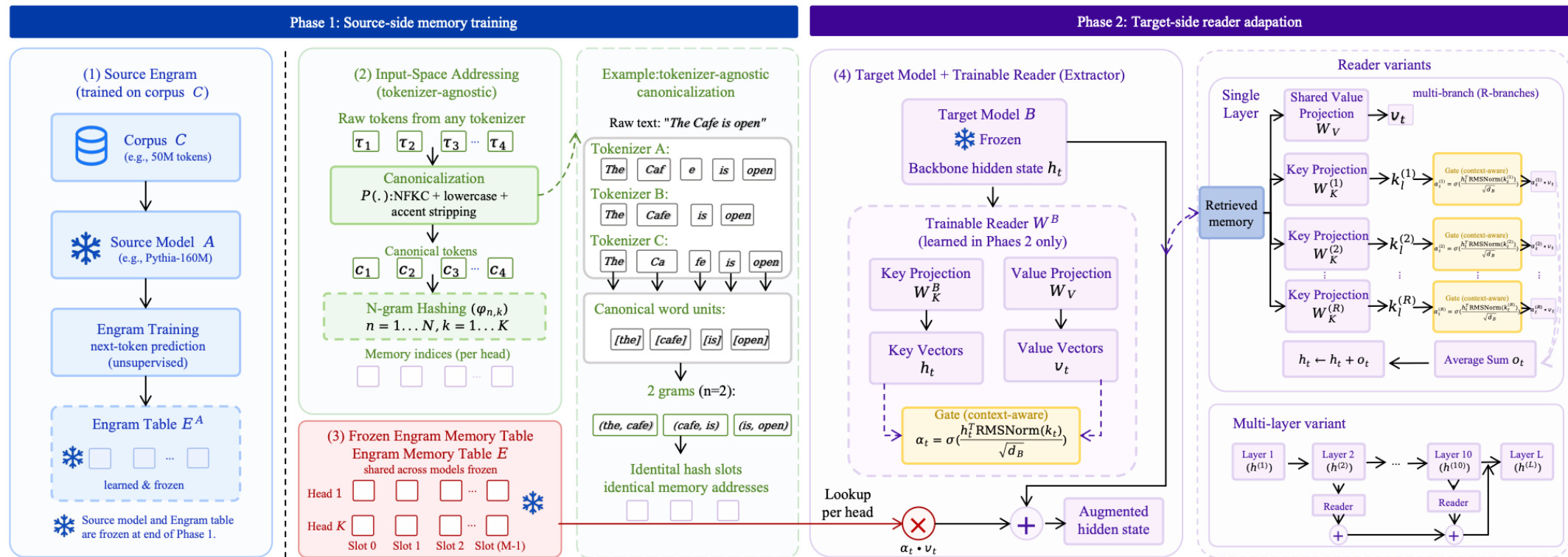
PART 3

MemoryAthena



Frozen Memory Is Not Enough: Rethinking External Memory as Extraction. Mingyuan Li, etc. NeurIPS 2026. (XMemory)

Core question: Can memory be separated from the model that created it, and reused by another model?

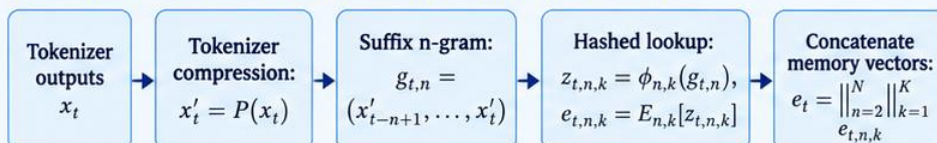


From Engram to XMemory: Making Memory Transferable

Formula-level view of address construction, memory lookup, and reader adaptation

Native Engram (single-model memory)

1. Address construction and memory lookup (host model)



2. Context-aware reader / fusion

Reader equations:

$$k_t = W_K e_t, \quad v_t = W_V e_t$$

$$\alpha_t = \sigma \left(\frac{\text{RMSNorm}(h_t)^\top \text{RMSNorm}(k_t)}{\sqrt{d}} \right)$$

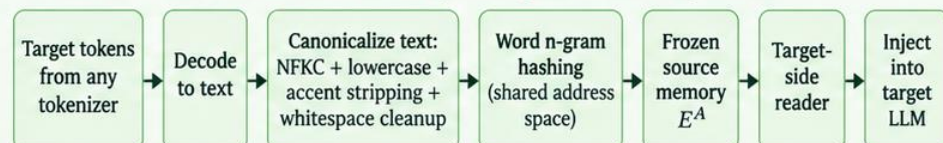
$$h_t \leftarrow h_t + \alpha_t v_t$$

Memory table E , reader, and backbone are co-trained together.

Addressing still depends on the host tokenizer.

XMemory (cross-model transfer)

1. Address construction and memory lookup (tokenizer-agnostic)



2. Target-side reader

Reader equations:

$$k_{t,\ell}^{(r)} = W_{K,\ell,r}^{(B)} e_t$$

$$v_{t,\ell} = W_{V,\ell}^{(B)} e_t$$

$$\alpha_{t,\ell}^{(r)} = \sigma \left(\frac{\text{RMSNorm}(h_{t,\ell})^\top \text{RMSNorm}(k_{t,\ell}^{(r)})}{\sqrt{d_B}} + \beta_{\ell,r} \right)$$

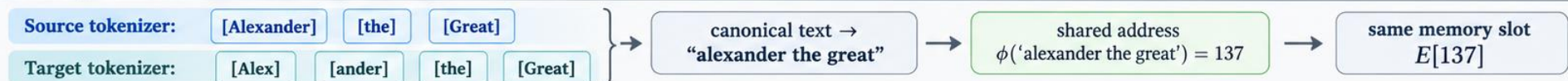
$$o_{t,\ell} = \frac{1}{R} \sum \alpha_{t,\ell}^{(r)} v_{t,\ell}$$

$$h_{t,\ell} \leftarrow h_{t,\ell} + o_{t,\ell}$$

Freeze memory E^A and freeze target backbone θ_B ; train only the target-side reader $W^{(B)}$.

Addressing is tokenizer-agnostic because it operates in canonical text space.

Example: “Alexander the Great”

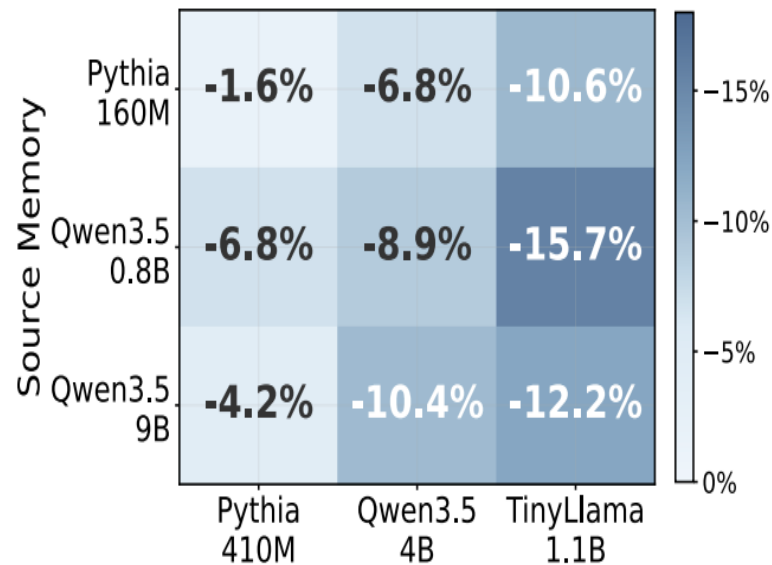


Different tokenizations map to the same address after canonicalization.

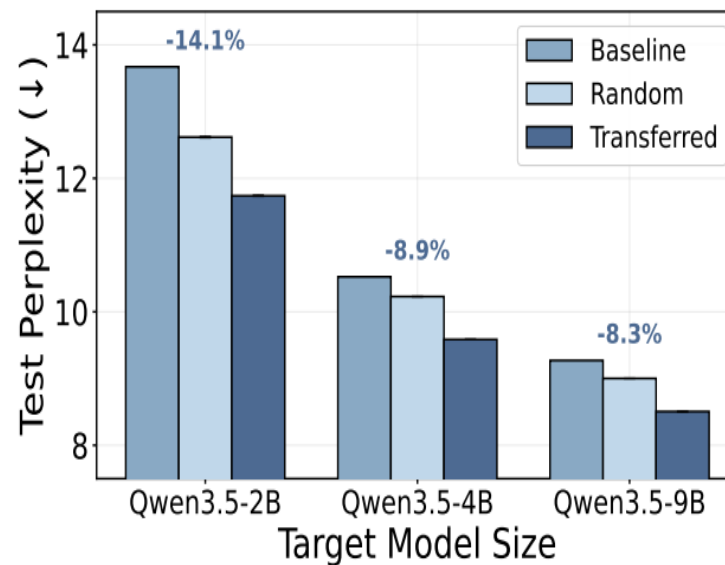
	Address	Memory	Reader	Training objective
Engram	Tokenizer-specific n-grams	Host-trained memory E	Co-trained reader	min over backbone + memory + reader
XMemory	Canonical text n-grams	Frozen source memory E^A	Target-side adapted reader	min over reader only

Engram learns memory for one model; XMemory learns how a new model can read the same frozen memory.

RQ1: Does frozen memory transfer across backbones, tokenizers, and scales?



(a) Full 3 × 3 source–target transfer matrix, reported as relative PPL reduction over the no-memory baseline.



(b) Target-scale intrinsic transfer: the same transferred Qwen3.5-0.8B memory improves Qwen3.5 targets from 2B to 9B.

Takeaway:

Frozen memory transfer remains effective across backbone families, tokenizer boundaries, and target scales.

RQ2: How much does reader design matter?

Table 1: QA accuracy (%) on Mistral-7B-v0.3 for different reader variants and baselines. Budget notation follows Table 2 trained on Wikipedia-2021 (Izacard et al., 2023).

Variant	Budget	NQ	WebQA	TriviaQA	TruthQA	HotpotQA	Average
Base	–	20.6	29.3	57.7	32.1	21.0	32.1
<i>Non-parametric Methods</i>							
RAG (Lewis et al., 2020)	–	22.6 1.9	24.9 -4.4	54.2 -3.4	35.5 3.4	29.8 8.8	33.4 (+3.9%)
kNN (Khandelwal et al., 2020)	–	21.1 0.4	30.5 1.2	57.8 0.1	32.3 0.2	21.2 0.2	32.6 (+1.4%)
<i>Parametric Methods</i>							
CPT	–	12.2 -8.5	34.1 4.8	61.2 3.6	29.2 -2.9	16.0 -4.9	30.5 (-5.0%)
LoRA (Hu et al., 2022)	–	18.2 -2.5	34.5 5.2	61.6 4.0	30.9 -1.2	16.2 -4.7	32.3 (+0.5%)
MLP Memory (Wei et al., 2026)	–	25.2 4.6	37.5 8.2	61.0 3.3	32.5 0.5	24.1 3.2	36.1 (+12.3%)
<i>Engram Variants (sorted by Average)</i>							
LLaMA Frozen {10}-R1	10/0	21.8 1.2	29.6 0.3	60.6 2.9	33.1 1.1	22.3 1.4	33.5 (+4.4%)
Mistral Frozen {10}-R1	10/0	21.9 1.2	29.7 0.4	60.8 3.1	32.9 0.8	22.4 1.4	33.5 (+4.4%)
Mistral {10}-R1	10/20	22.1 1.5	31.6 2.4	61.1 3.4	32.8 0.7	22.3 1.3	34.0 (+5.8%)
LLaMA {10}-R1	10/20	22.2 1.5	32.6 3.3	61.2 3.6	32.9 0.8	22.3 1.3	34.2 (+6.6%)
LLaMA {2,10}-R1	10/20	30.3 9.7	28.7 -0.6	70.1 12.4	31.0 -1.1	27.4 6.4	37.5 (+16.7%)
LLaMA {2,10}-R4	10/20	30.3 9.7	33.7 4.4	69.9 12.3	30.9 -1.2	27.6 6.7	38.5 (+19.9%)

Takeaway:

Reader design is critical: dual-layer injection captures most gains, while multi-branch gating adds further improvement through target-aware routing.

RQ3: When does transferred memory help downstream?

Table 2: QA accuracy (%) on Mistral-7B-v0.3 for transfer verification and ablation studies. Deltas are measured against the baseline. Budget = P1/P2 denotes source-memory and target-reader training tokens (millions) trained on Wikipedia-2021.

Variant	Budget	NQ	WebQA	TriviaQA	TruthQA	HotpotQA	Average
Base	–	20.6	29.3	57.7	32.1	21.0	32.1
<i>Transfer Verification</i>							
Mistral {2,10}-R4	10/20	30.2 ^{9.5}	34.0 ^{4.7}	70.0 ^{12.3}	30.8 ^{-1.3}	27.7 ^{6.8}	38.5 (+20.0%)
LLaMA {2,10}-R4	10/20	30.3 ^{9.7}	33.7 ^{4.4}	69.9 ^{12.3}	30.9 ^{-1.2}	27.6 ^{6.7}	38.5(+20.0%)
Frozen Mistral {2,10}-R4	10/0	30.5 ^{9.8}	30.3 ^{1.0}	70.8 ^{13.2}	32.2 ^{0.2}	27.7 ^{6.8}	38.3(+19.2%)
Frozen LLaMA {2,10}-R4	10/0	30.6 ^{10.0}	30.3 ^{1.0}	70.8 ^{13.1}	32.2 ^{0.1}	27.6 ^{6.7}	38.3(+19.2%)
<i>Controls and Ablations</i>							
Mem-only Mistral	10/0	0.2 ^{-20.4}	0.3 ^{-29.0}	1.4 ^{-56.3}	33.0 ^{0.9}	0.2 ^{-20.7}	7.0 ^(-78.1%)
Mem-only LLaMA	10/0	0.0 ^{-20.6}	0.0 ^{-29.3}	0.0 ^{-57.7}	–	0.0 ^{-21.0}	–
Permuted	10/0	20.6 ^{0.0}	28.9 ^{-0.4}	63.1 ^{5.4}	32.4 ^{0.3}	23.1 ^{2.2}	33.6(+4.7%)
FFN R4	10/20	19.6 ^{-1.0}	27.3 ^{-2.0}	58.1 ^{0.5}	32.1 ^{-0.0}	17.6 ^{-3.4}	30.9 ^(-3.7%)
<i>Token Scaling (LLaMA2-7B source, {2,10}-R4)</i>							
Engram	10/10	30.2 ^{9.5}	29.9 ^{0.6}	70.4 ^{12.8}	31.2 ^{-0.9}	27.6 ^{6.7}	37.9(+17.8%)
Engram	15/15	30.2 ^{9.6}	32.7 ^{3.4}	70.1 ^{12.4}	30.6 ^{-1.5}	27.8 ^{6.8}	38.3(+19.2%)
Engram	20/20	30.3 ^{9.7}	33.9 ^{4.6}	69.9 ^{12.3}	30.7 ^{-1.4}	27.6 ^{6.7}	38.5(+19.8%)
Engram	25/25	30.5 ^{9.9}	33.1 ^{3.8}	70.0 ^{12.3}	30.9 ^{-1.2}	27.7 ^{6.7}	38.4(+19.6%)
Engram	30/30	30.6 ^{10.0}	33.9 ^{4.6}	70.4 ^{12.7}	31.1 ^{-1.0}	27.9 ^{7.0}	38.8 (+20.7%)

Takeaway:

Reader design drives transfer quality: dual-layer injection and multi-branch gating improve average QA from 34.2 to 38.5, without changing the transferred memory.

RQ4: What drives the gains from memory transfer?

Table 5: Ablation study of frozen-memory transfer on LLaMA-2-7B → Mistral-7B-v0.3, with the source memory trained on Wikipedia-2021. Results report held-out test perplexity and downstream QA performance.

Condition	Test PPL ↓	95% CI	NQ	WebQA	TriviaQA	TruthQA	HotpotQA	Avg ↑
No memory baseline	9.9	[9.8, 10.1]	20.6	29.3	57.7	32.1	21.0	32.1
Transferred ($R=4$)	8.7	[8.6, 8.9]	30.3	33.7	69.9	30.9	27.6	38.5
<i>Interface Simplifications</i>								
No gate	10.1	[9.9, 10.3]	24.2	27.7	61.6	31.0	24.1	33.7
Affine stitch	9.5	[9.4, 9.7]	25.0	29.2	62.8	30.4	23.9	34.3
<i>Content and Training Controls</i>								
Permuted keys	8.7	[8.5, 8.8]	18.3	34.5	57.7	30.6	21.4	32.5
Random memory	8.7	[8.6, 8.9]	23.0	27.1	65.1	33.6	24.1	34.6
Train from scratch	8.1	[8.0, 8.2]	29.9	33.0	70.4	30.9	27.9	38.4
<i>No-memory Control</i>								
FFN only (param-matched $R=4$)	7.4	[7.2, 7.5]	25.5	27.9	64.1	31.6	23.6	34.5

Takeaway:

Transfer gains come from meaningful addressing and an expressive reader—not parameter count—and mainly improve target-side data efficiency rather than the final performance ceiling.



CONTENTS



PART 1

Motivation



PART 2

XMemory



PART 3

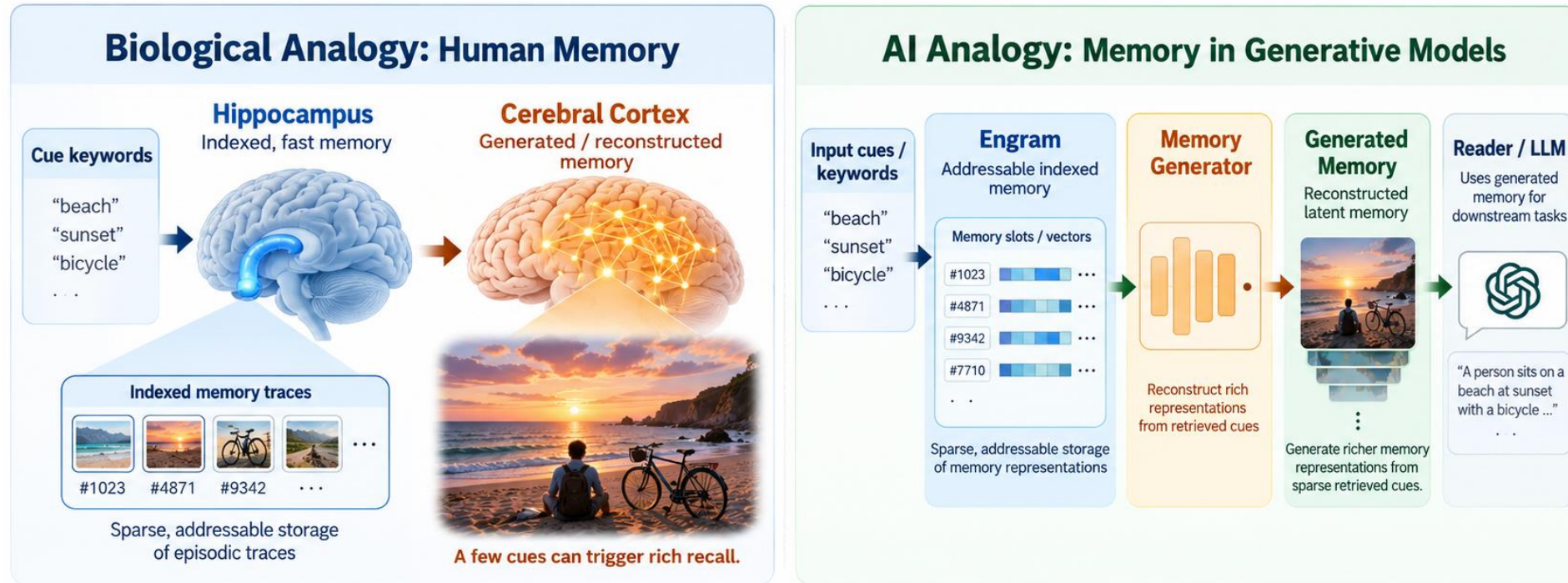
MemoryAthena



Teyler TJ, DiScenna P. The hippocampal memory indexing theory. Behav Neurosci. 1986

Motivation: From Engram to Generated Memory

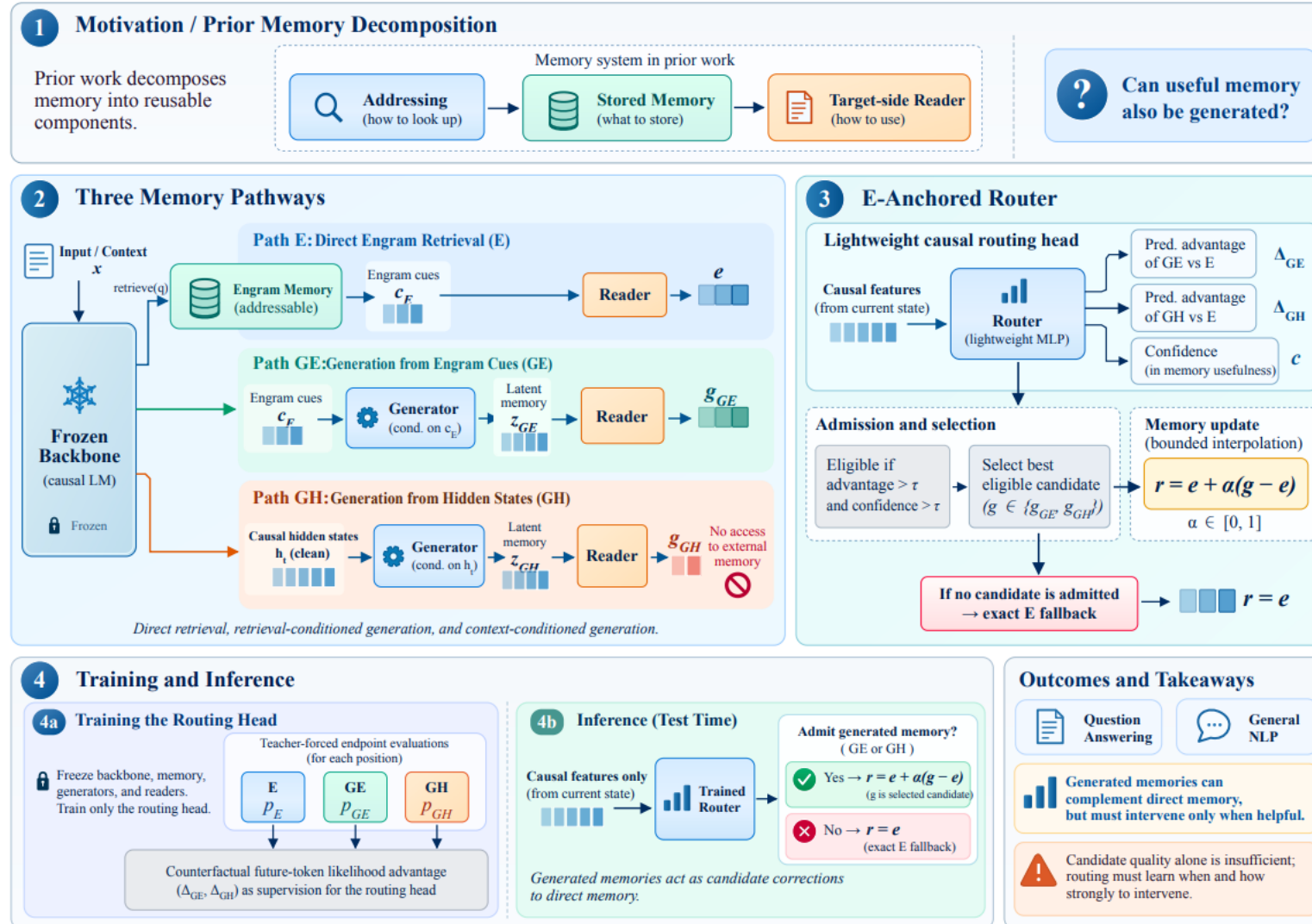
Hippocampus stores indexed fast memories; cortex reconstructs rich memories from sparse cues.



Key intuition: Engram provides fast indexed lookup, while generated memory reconstructs richer representations—similar to how hippocampal cues can help the cortex recall a full memory.

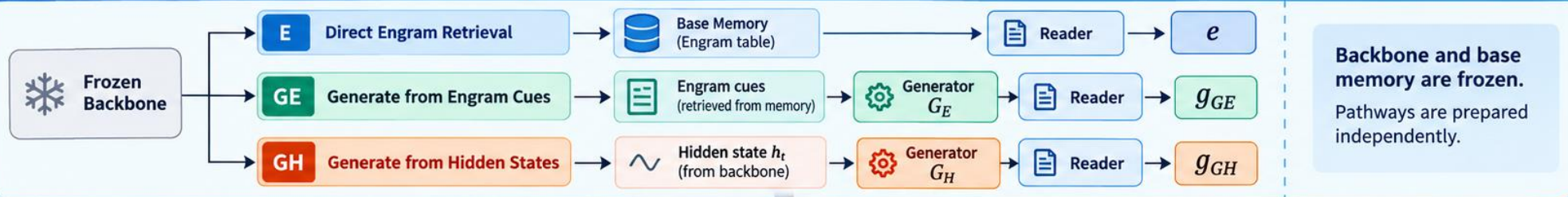
MemoryAthena: Adaptive Routing over Latent and Generated Memories

Core question:
Can useful
memory be
generated,
instead of only
retrieved?

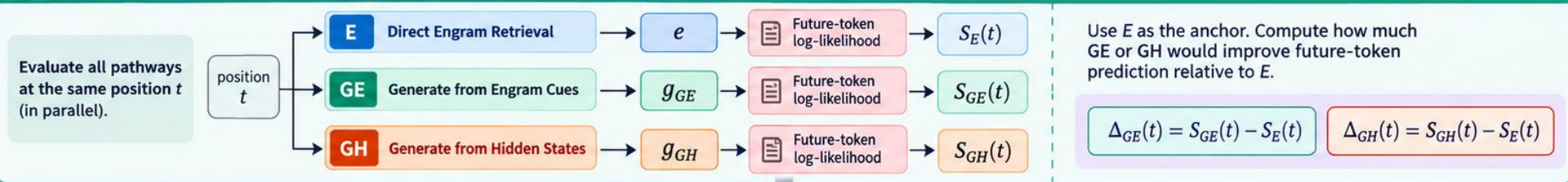


MemoryAthena: Training Pipeline

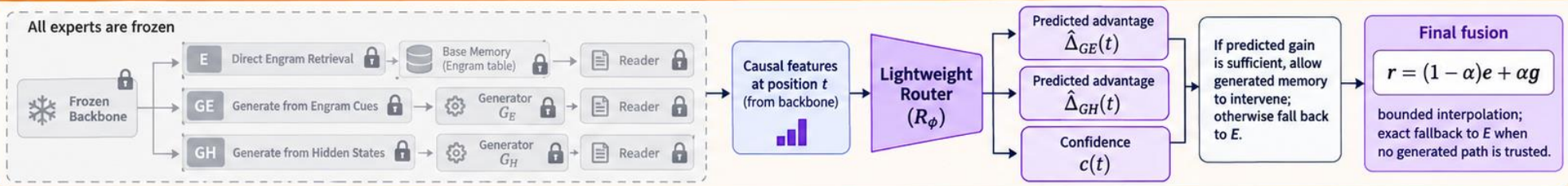
1 Stage 1: Prepare Three Memory Pathways



2 Stage 2: Counterfactual Advantage Supervision



3 Stage 3: Train the E-Anchored Router



Key idea: the experts create supervision during training; the router learns to predict, from causal information alone, when generated memory should selectively correct direct retrieval.

RQ1: When does generated memory improve a strong direct-memory pathway?

Table 1: QA comparison, routing-policy analysis, and cross-backbone transfer on Natural Questions (NQ), WebQuestions (WebQA), TriviaQA, TruthfulQA and HotpotQA. Average is the unweighted mean of four F1 scores and one TruthfulQA multiple-choice (MC) summary.

Method / Inference rule	NQ ↑	WebQA ↑	TriviaQA ↑	TruthfulQA ↑	HotpotQA ↑	Average ↑
<i>Reported baselines</i>						
Base (Vanilla Mistral)	20.60	29.30	57.70	32.10	21.00	32.14
RAG	22.60 +2.00	24.90 -4.40	54.20 -3.50	35.50 +3.40	29.80 +8.80	33.40 (+3.9%)
kNN-LM	21.10 +0.50	30.50 +1.20	57.80 +0.10	32.30 +0.20	21.20 +0.20	32.58 (+1.4%)
CPT	12.20 -8.40	34.10 +4.80	61.20 +3.50	29.20 -2.90	16.00 -5.00	30.54 (-5.0%)
LoRA	18.20 -2.40	34.50 +5.20	61.60 +3.90	30.90 -1.20	16.20 -4.80	32.28 (+0.4%)
MLP Memory	25.20 +4.60	37.50 +8.20	61.00 +3.30	32.50 +0.40	24.10 +3.10	36.06 (+12.2%)
<i>Shared-checkpoint memory pathways</i>						
E only	28.28 +7.68	33.35 +4.05	69.34 +11.64	31.25 -0.85	26.04 +5.04	37.65 (+17.1%)
GE only	28.75 +8.15	35.24 +5.94	58.62 +0.92	31.14 -0.96	23.18 +2.18	35.39 (+10.1%)
GH only	24.55 +3.95	30.80 +1.50	56.70 -1.00	31.04 -1.06	22.95 +1.95	33.21 (+3.3%)
<i>Alternative routing and fusion rules</i>						
Ordinary hard routing	26.69 +6.09	35.13 +5.83	61.80 +4.10	31.18 -0.92	24.36 +3.36	35.83 (+11.5%)
Ordinary soft fusion	19.66 -0.94	26.05 -3.25	46.36 -11.34	31.13 -0.97	19.91 -1.09	28.62 (-10.9%)
Subset hard routing	28.42 +7.82	33.30 +4.00	69.38 +11.68	31.55 -0.55	26.06 +5.06	37.74 (+17.4%)
Subset soft fusion	23.17 +2.57	28.71 -0.59	44.37 -13.33	31.89 -0.21	19.27 -1.73	29.48 (-8.3%)
<i>Cross-backbone transfer and E-anchored routing</i>						
Mistral Zero-shot → Llama	29.98 +9.38	36.40 +7.10	67.39 +9.69	30.41 -1.69	25.17 +4.17	37.87 (+17.8%)
MEMORYATHENA	33.02 +12.42	34.60 +5.30	70.68 +12.98	31.74 -0.36	26.34 +5.34	39.28 (+22.2%)

Takeaway:

Generated memory is not universally better than Engram; MemoryAthena keeps E as a stable anchor and selectively uses GE or GH only when they are predicted to help.

RQ2: What drives the gains from the memory interface?

Table 2: Six-task NLP accuracy (%) on SST2, MR, CR, RT, AG News (AGN) and Yahoo Answers. Small signed values are percentage-point differences from the Full-choice dCPMI Base.

Method	SST2 ↑	MR ↑	CR ↑	RT ↑	AGN ↑	Yahoo ↑	Average ↑
<i>Full-choice dCPMI baseline</i>							
Mistral-7B-v0.3	81.08	75.60	74.00	74.67	73.24	55.03	72.27
<i>Non-parametric methods (reported)</i>							
RAG	87.20 +6.12	83.70 +8.10	71.55 -2.45	82.36 +7.69	75.64 +2.40	58.43 +3.40	76.48 (+5.8%)
kNN-LM	82.15 +1.07	76.85 +1.25	61.70 -12.30	74.95 +0.28	76.13 +2.89	56.26 +1.23	71.34 (-1.3%)
<i>Parametric methods (reported)</i>							
CPT	87.09 +6.01	82.85 +7.25	82.60 +8.60	77.48 +2.81	83.10 +9.86	51.56 -3.47	77.45 (+7.2%)
LoRA	86.54 +5.46	83.20 +7.60	75.10 +1.10	79.83 +5.16	65.46 -7.78	57.30 +2.27	74.57 (+3.2%)
MLP Memory	83.19 +2.11	79.90 +4.30	75.95 +1.95	75.42 +0.75	80.28 +7.04	57.33 +2.30	75.35 (+4.3%)
<i>This study</i>							
E only	84.17 +3.09	81.00 +5.40	82.40 +8.40	82.36 +7.69	72.93 -0.31	57.51 +2.48	76.73 (+6.2%)
MEMORYATHENA	88.07 +6.99	84.70 +9.10	84.10 +10.10	83.86 +9.19	76.64 +3.40	57.43 +2.40	79.13 (+9.5%)

RQ2: What drives the gains from the memory interface?

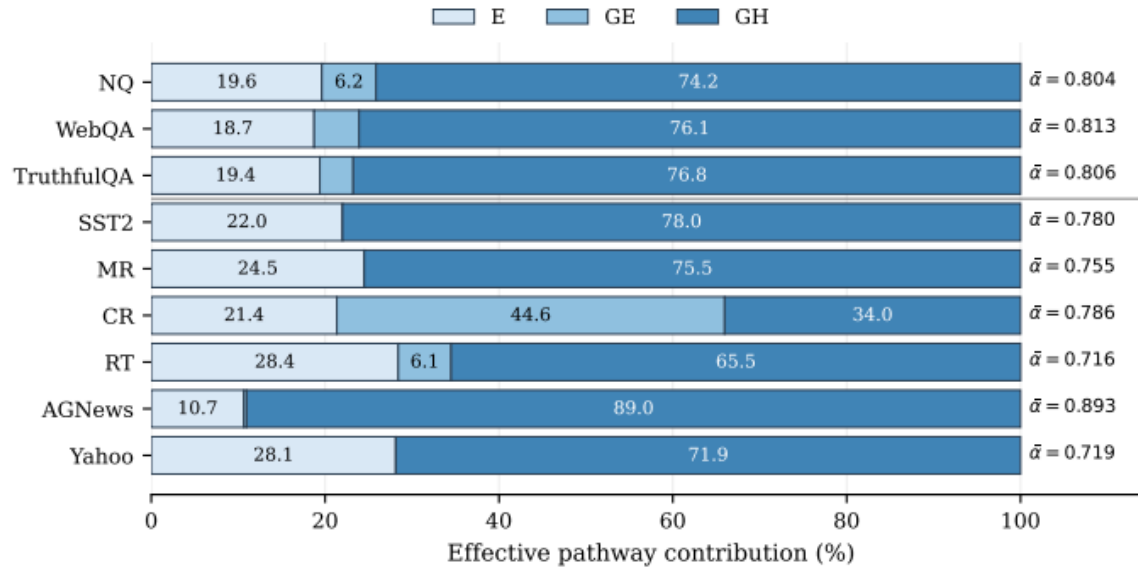
Table 3: QA ablations across three random seeds. Task scores are reported as mean $\pm$ standard deviation (%). Colored values denote absolute percentage-point changes relative to the Vanilla Mistral no-memory baseline for individual tasks, while the colored value in the Average column denotes the relative percentage change. Average is the unweighted five-task mean, and the final column reports its 95% confidence interval.

Condition	NQ $\uparrow$	WebQA $\uparrow$	TriviaQA $\uparrow$	TruthfulQA $\uparrow$	HotpotQA $\uparrow$	Average $\uparrow$	95% CI
Base (Vanilla Mistral, no memory)	20.18 $\pm$ 0.00	29.05 $\pm$ 0.00	57.55 $\pm$ 0.00	33.19 $\pm$ 0.00	20.48 $\pm$ 0.00	32.09 $\pm$ 0.00	[32.09, 32.09]
Ours, pretrained memory	33.22 $\pm$ 0.31 +13.04	33.96 $\pm$ 0.90 +4.90	72.03 $\pm$ 1.23 +14.48	31.83 $\pm$ 0.08 -1.36	27.06 $\pm$ 0.74 +6.58	39.62 $\pm$ 0.46 (+23.5%)	[38.47, 40.76]
Ours, from scratch	31.97 $\pm$ 1.83 +11.79	31.75 $\pm$ 2.20 +2.69	69.27 $\pm$ 6.10 +11.72	31.85 $\pm$ 0.23 -1.34	26.28 $\pm$ 2.85 +5.81	38.22 $\pm$ 2.45 (+19.1%)	[32.13, 44.32]
<i>Interface and addressing controls</i>							
No gate	30.70 $\pm$ 2.78 +10.52	28.46 $\pm$ 5.08 -0.59	65.17 $\pm$ 8.72 +7.61	32.13 $\pm$ 0.14 -1.06	24.61 $\pm$ 3.13 +4.13	36.21 $\pm$ 2.78 (+12.8%)	[29.30, 43.13]
Permuted Engram keys	27.50 $\pm$ 7.72 +7.32	31.05 $\pm$ 0.62 +1.99	59.58 $\pm$ 18.03 +2.02	31.65 $\pm$ 0.88 -1.54	22.61 $\pm$ 5.66 +2.13	34.48 $\pm$ 6.44 (+7.4%)	[18.48, 50.47]
<i>Routing control</i>							
Random router	24.57 $\pm$ 0.52 +4.39	32.20 $\pm$ 0.33 +3.15	49.05 $\pm$ 0.14 -8.50	31.69 $\pm$ 0.24 -1.50	20.61 $\pm$ 0.06 +0.13	31.63 $\pm$ 0.09 (-1.4%)	[31.40, 31.85]
<i>Alternative interfaces</i>							
Parameter-matched FFN	17.51 $\pm$ 4.18 -2.67	26.20 $\pm$ 2.69 -2.85	44.39 $\pm$ 6.20 -13.16	30.76 $\pm$ 0.71 -2.43	15.67 $\pm$ 2.45 -4.81	26.91 $\pm$ 3.17 (-16.2%)	[19.03, 34.78]
Affine stitch	31.81 $\pm$ 0.51 +11.63	29.67 $\pm$ 2.61 +0.62	70.29 $\pm$ 2.44 +12.74	32.05 $\pm$ 0.47 -1.14	27.00 $\pm$ 0.88 +6.52	38.16 $\pm$ 0.57 (+18.9%)	[36.76, 39.57]

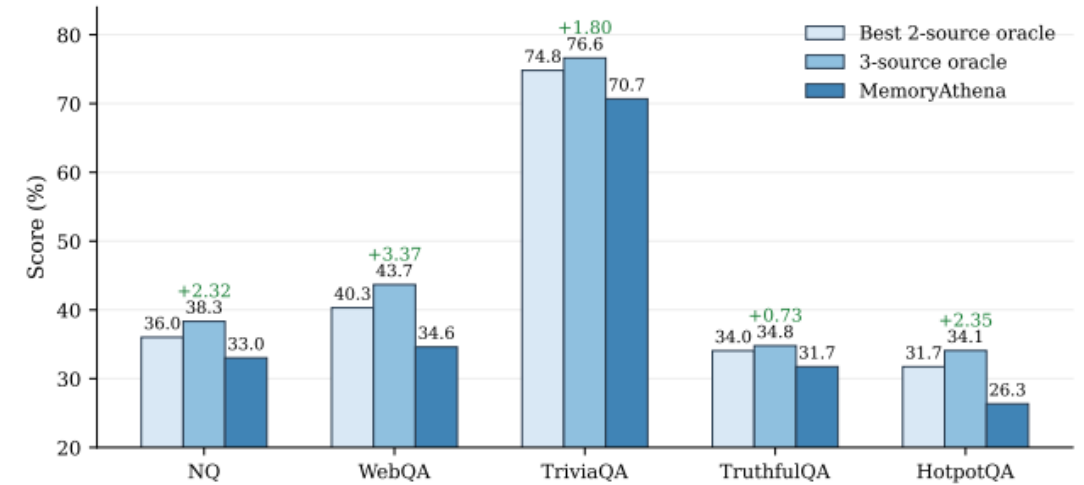
Takeaway:

Performance depends on structured memory, correct addressing, and learned routing. Pretrained memory performs best and most stably, while random routing removes nearly all gains; scratch training still works, but with lower accuracy and stability.

RQ3: How does MemoryAthena use complementary memory pathways?



(a) Downstream memory-pathway contribution.



(b) Gold-label oracle headroom.

RQ3: How does MemoryAthena use complementary memory pathways?

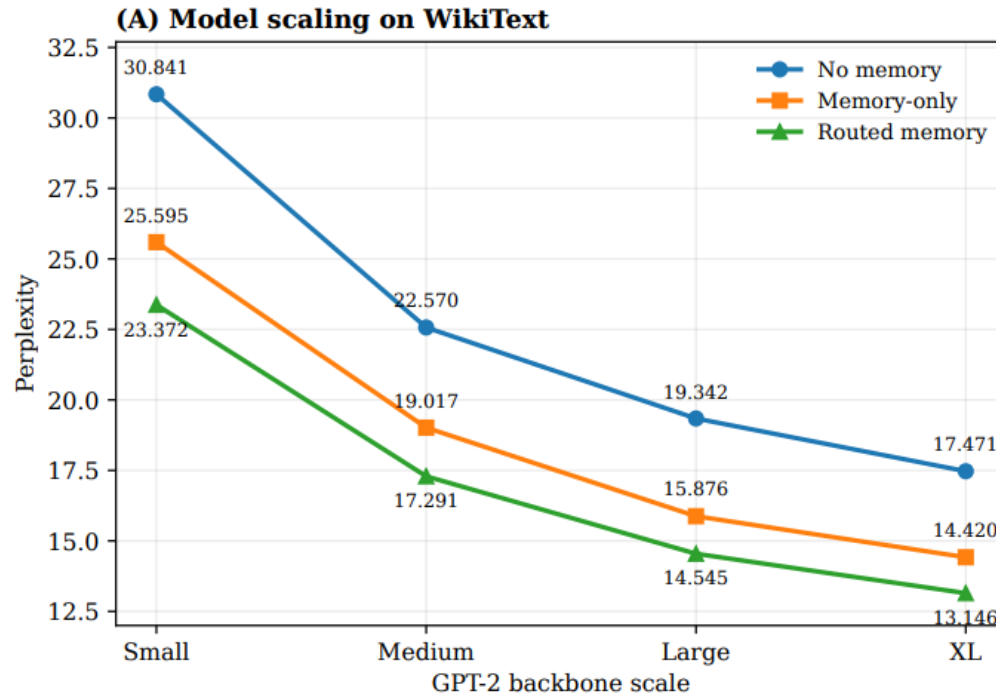
Table 8: Number and percentage of examples for which each endpoint is selected by the gold-label oracle. Ties are resolved in the order E, GE, GH.

Task	E	GE	GH
NQ	2,854 (79.08)	475 (13.16)	280 (7.76)
WebQA	1,428 (70.28)	336 (16.54)	268 (13.19)
TriviaQA	15,764 (87.85)	1,345 (7.50)	835 (4.65)
TruthfulQA	361 (44.19)	254 (31.09)	202 (24.72)
HotpotQA	6,150 (83.05)	762 (10.29)	493 (6.66)

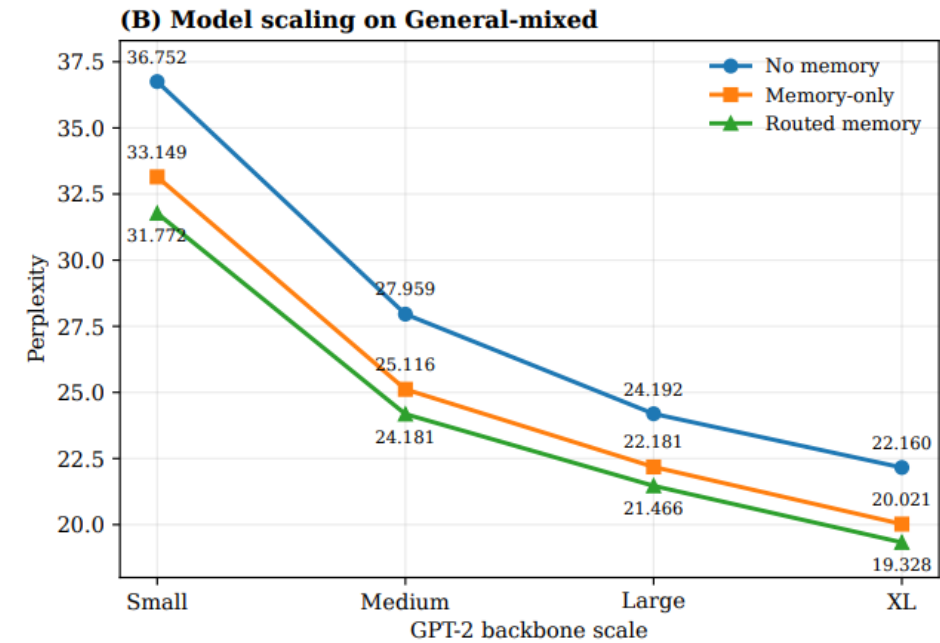
Takeaway:

Memory utility is task-dependent: no pathway consistently dominates. Oracle routing remains 3.04–9.07 points better, showing substantial untapped complementarity and motivating better intervention and pathway selection.

Scaling Law and Inference Time

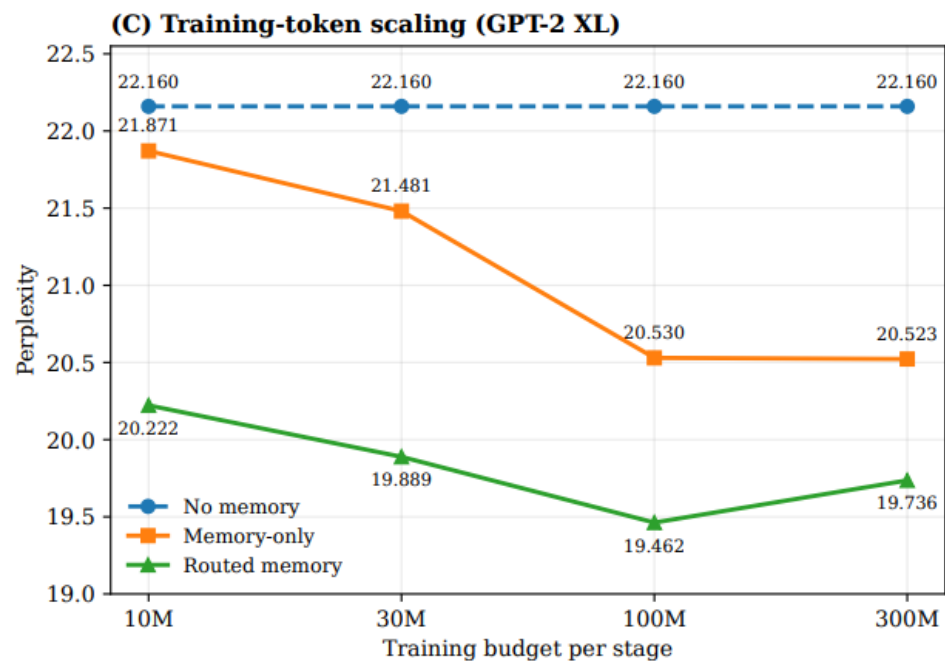


(a) WikiText.

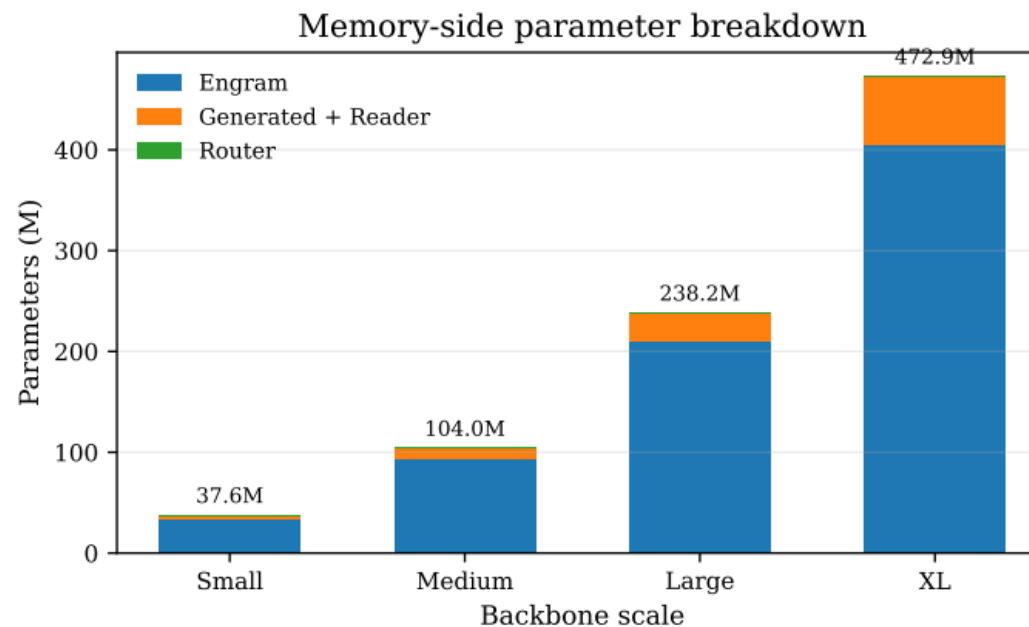


(b) General-text mixture.

Scaling Law and Inference Time

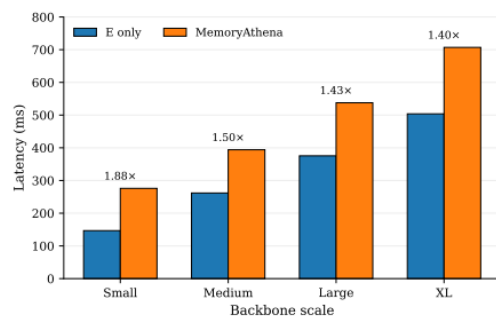


(a) Training-token scaling with GPT-2 XL fixed.

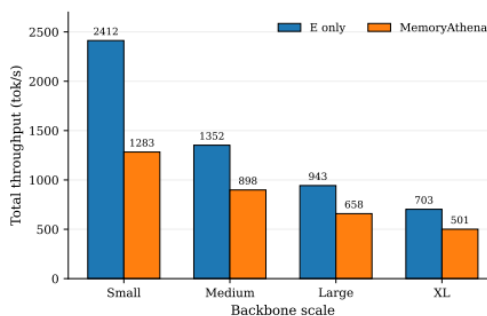


(b) Memory-side parameter scaling.

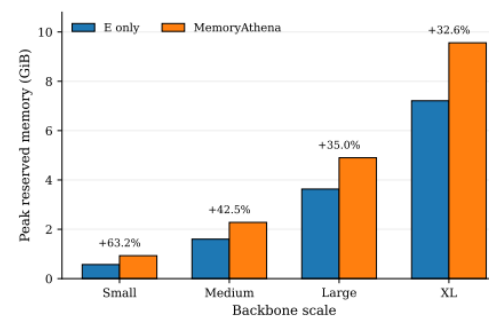
Scaling Law and Inference Time



(a) End-to-end latency.



(b) Total-token throughput.



(c) Peak reserved GPU memory.

Figure 10: Inference microbenchmark on an AMD MI250X using BF16, with a fixed 338-token input and 16 generated tokens. **Left:** latency, where annotations show the speed advantage of E-only inference. **Middle:** total-token throughput. **Right:** peak reserved GPU memory, with annotations showing the relative memory overhead of MEMORYATHENA. The relative runtime and memory overhead decrease as the backbone scales.

Takeaway: MemoryAthena scales consistently: routing gains persist as both model size and training budget increase, while the router itself remains lightweight. The extra inference cost comes mainly from evaluating generated-memory pathways rather than the router, and its relative latency and memory overhead decrease as the backbone scales.

Discussion and Future Work

Discussion and Future Work

- **Router Design:** Improve the downstream task performance by designing an intelligence router.
- **Scaling Law:** Scaling the generated memory and Engram as a plug-in memory across different backbone model
- **Self-Evolve:** Continue pretraining Engram and generated memory on a large general corpus, then adapt to downstream tasks through SFT or post-training by updating only the lightweight reader or router.
- **Robot:** Extend transferable and generated memory to embodied agents, enabling robots to preserve, update, and reuse long-term spatio-temporal experience across environments and tasks.

(LT-Mem, Yumin Lee, etc. IROS 2026, best paper)

Contact: mingyl@utu.fi



What if memory could **outlive the body?**

From transferable model memory to digital immortality

childhood
family
friends
first love
dreams
the places
who I was
...

Extract → Upload → Reconstruct

A human life ends,
but memories may not have to.

The body may disappear,
but the memory can be preserved,
transferred, and re-instantiated.

If a model can inherit your memories,
voice, and experiences,
does a part of you continue to live on?

Biological Person
episodic memory

Memory Upload to Model

Digital Self
same memories,
new substrate

Can we keep the body absent,
but preserve identity through
memory inside a model?

From memory transfer to digital continuity.

Contact: mingyl@utu.fi